Lecture 10: CLT, Standard Error, Pandas

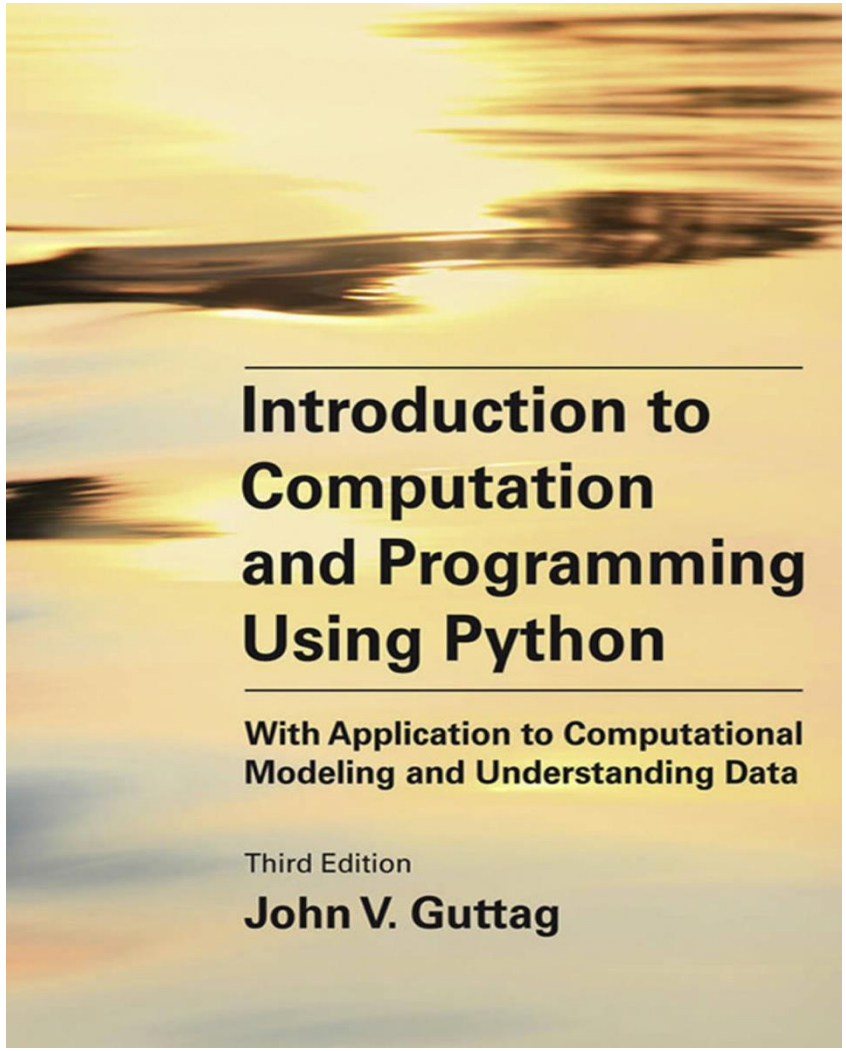
(download slides and .py files to follow along)

John Guttag

MIT Department of Electrical Engineering and Computer Science

Assigned Reading

- Today
  - Chapters 19, 23
- Next lecture
  - Chapter 21-22



Upcoming Schedule

- April 6, recitation (Pandas)
- April 8, microquiz covering lectures 9 - 10
- Three more lectures
  - April 13: Statistics and how to lie with them
  - April 15: Introduction to ML
  - April 23: More ML, course wrap up
- What else is left
  - April 27: Recitation
  - April 29: Last quiz, covering lectures 11 - 13
  - May 5: PS 5 due

Today

- Distributions and sampling
- Central limit theorem
- Quick look at Pandas
- Microquiz starting at noon

Concept of a Probability Distribution (from last lecture)

- As $\boldsymbol{n}$ increases:

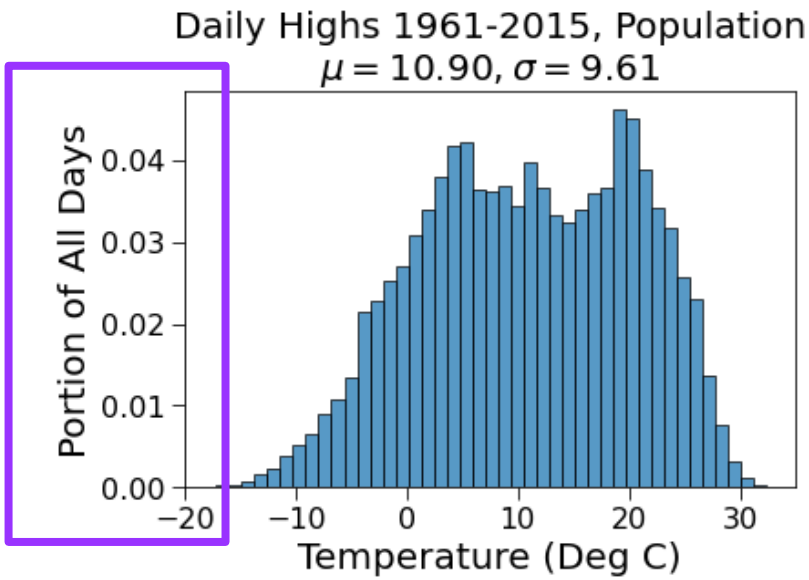
$$\frac{(\#\textbf{of } s_j = x_i)}{n} \quad \textbf{approaches} \quad \Pr(X = x_i)$$

- Rescale plot to represent **probability of landing in any bin**
  - **All probabilities sum to 1**

ith possible outcome value

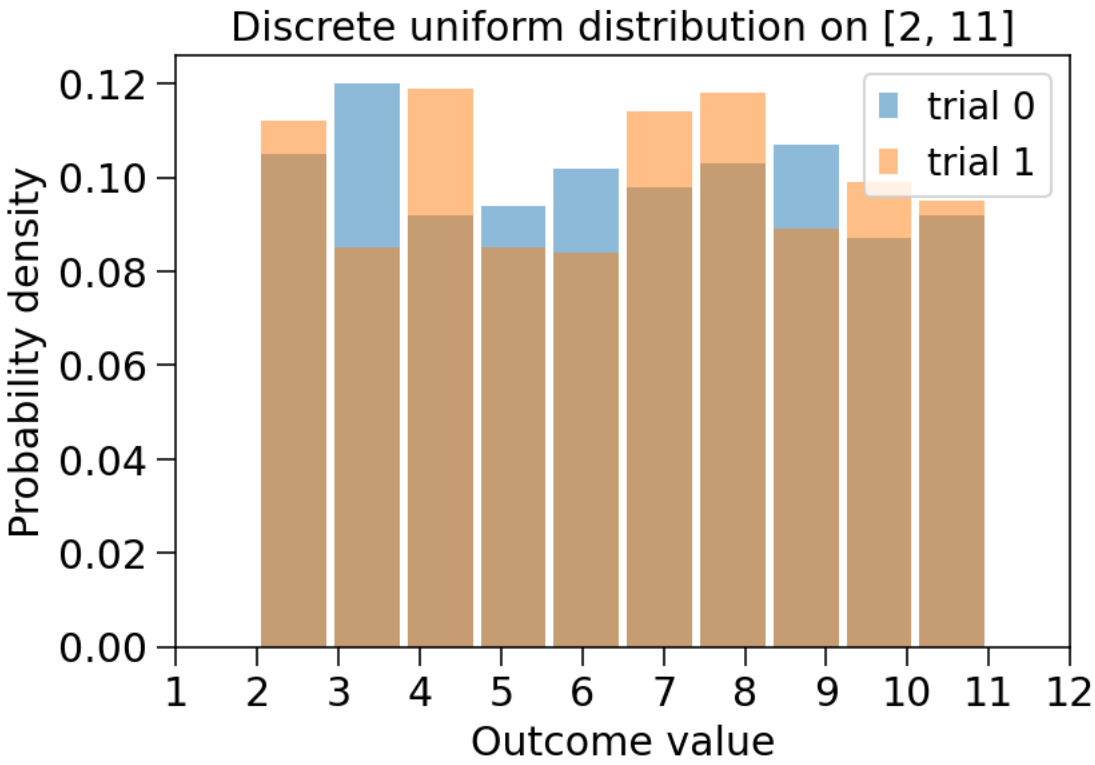
$$\mu = \sum x_i \cdot \Pr(X = x_i)$$

$$\sigma^2 = \sum (x_i - \mu)^2 \cdot \Pr(X = x_i)$$



Uniform Distribution (discrete)

- Equal probability for all **integer** outcomes within lower and upper bounds $[\boldsymbol{a}, \boldsymbol{b}]$



Uniform Distribution: Examples

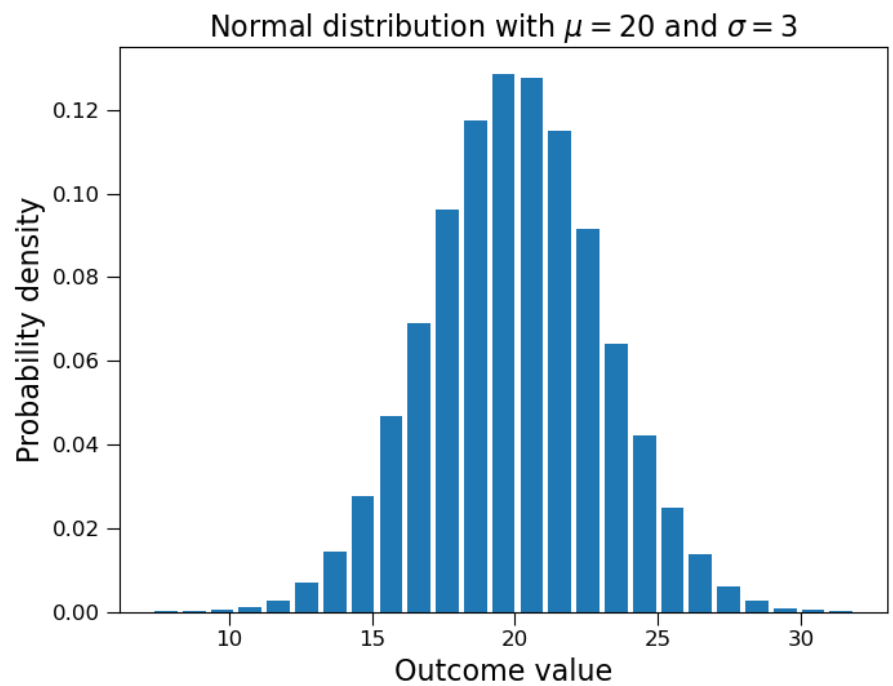
- Coin flipping
- Dice rolling
- Roulette pocket (not return)
- Location of territorial animals
  - Social distancing?

Normal Distribution

- Parameterized by mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$

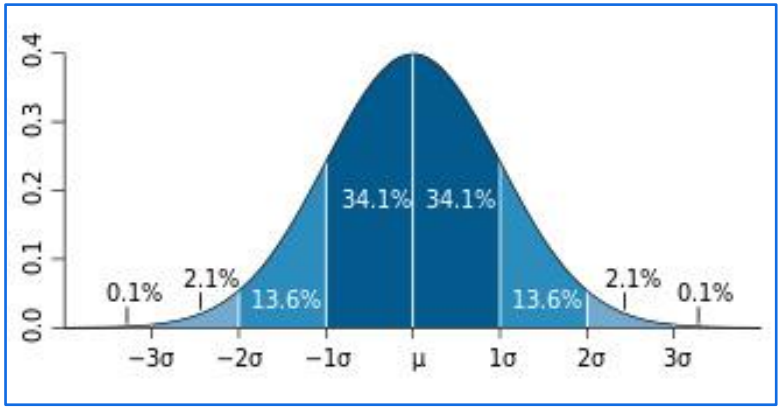
```
mu, sigma = 20, 3
num_samples = 100_000
num_bins = 25
normal_vals = [random.gauss(20, 3) for _ in range(num_samples)]
counts, bins = np.histogram(normal_vals, bins = num_bins)
prob_dist = counts/num_samples
x_vals = [(bins[i] + bins[i-1])/2 for i in range(1, len(bins))]
plt.bar(x_vals, prob_dist)
```

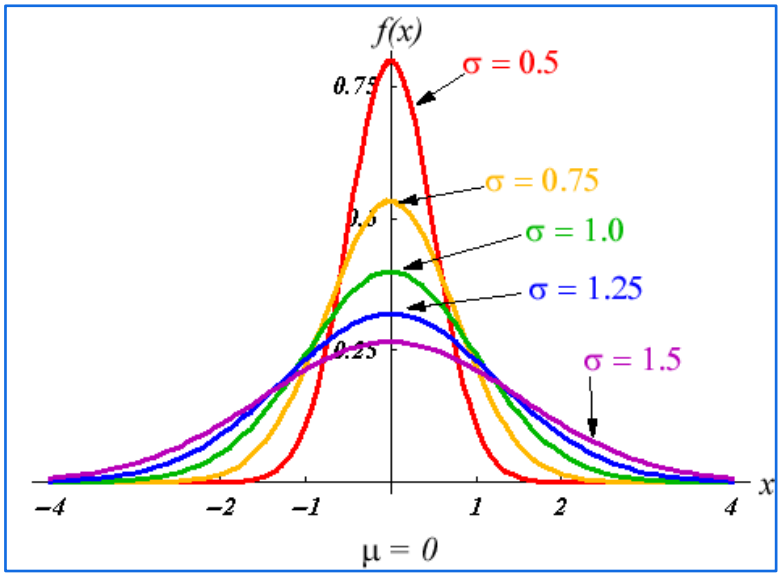
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



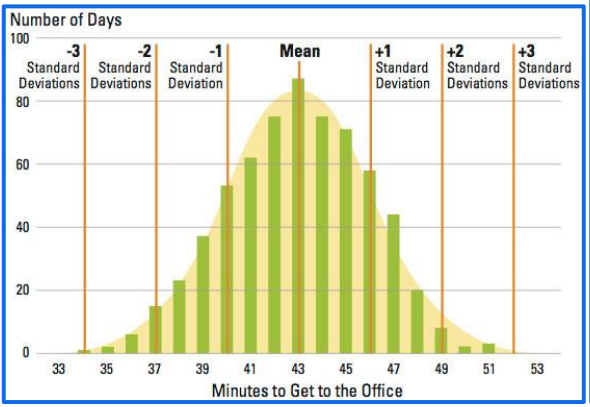
Everybody Loves a Normal Distribution

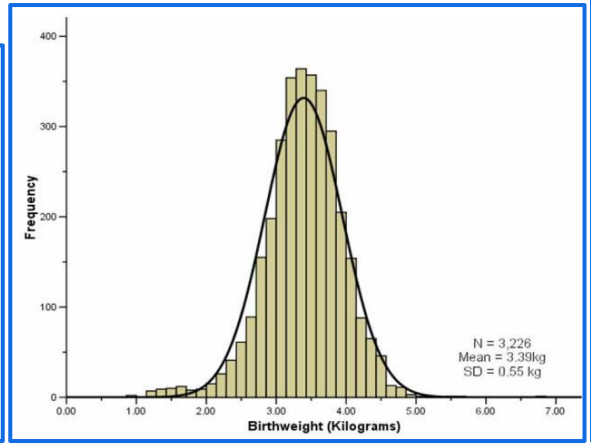
- Nice mathematical properties
  - Symmetric around mean
  - Mean is also mode and median
  - Area under curve is 1
  - Its density is infinitely differentiable
  - It is unimodal – its first derivative is positive to the left of the mean, negative to the right of the mean and zero only at the mean

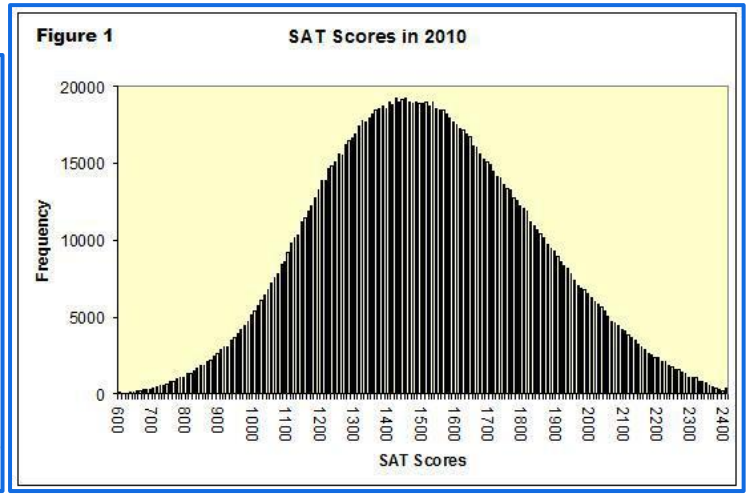


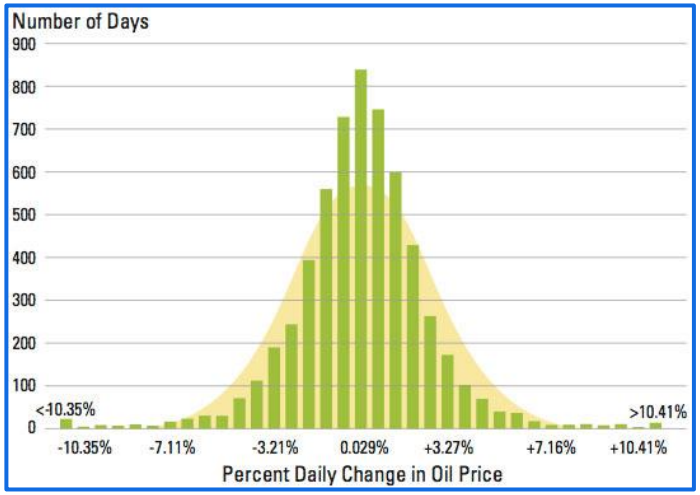


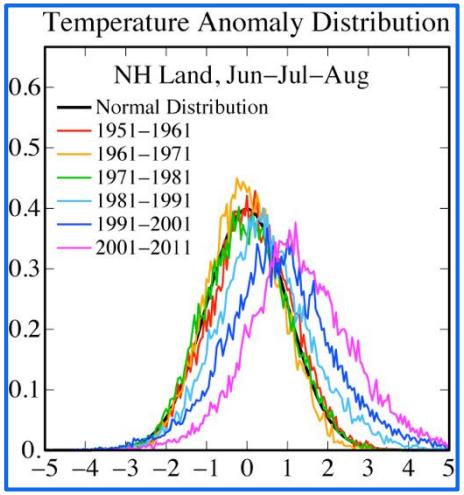
Most Importantly They Occur a Lot!

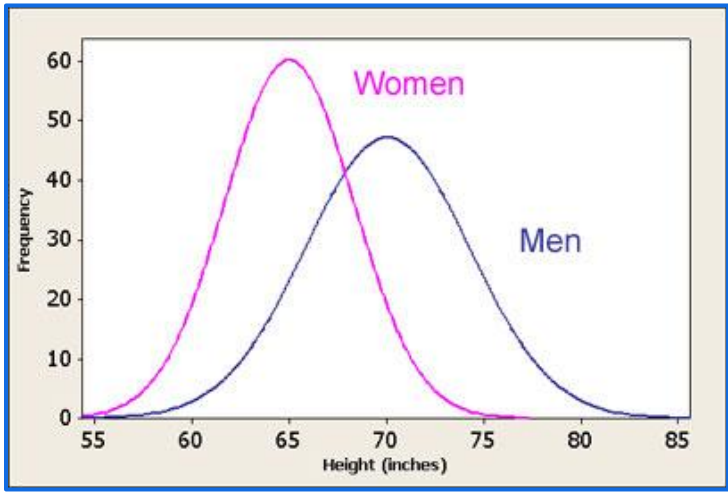












Exponential Distribution

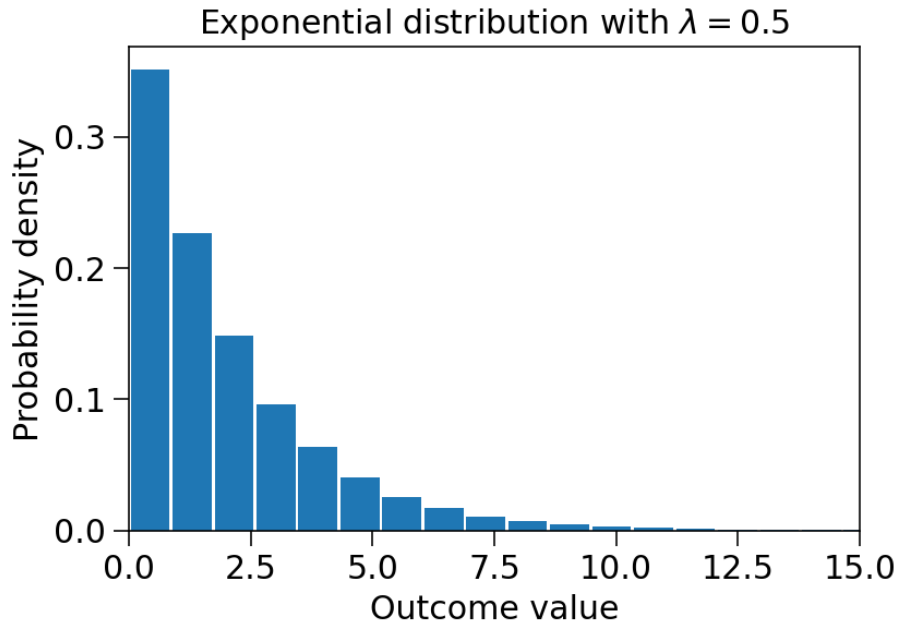
- Describes time between events, where events happen continuously and independently at a rate $\boldsymbol{\lambda}$
  - Events occur every $\frac{1}{\lambda}$ steps on average

If 10 people arrive per hour, then we expect 1/10 of an hour = 6 min between arrivals

$$f(x) = \lambda e^{-\lambda x}$$

$$\mu = \frac{1}{\lambda}$$

$$\sigma^2 = \frac{1}{\lambda^2}$$



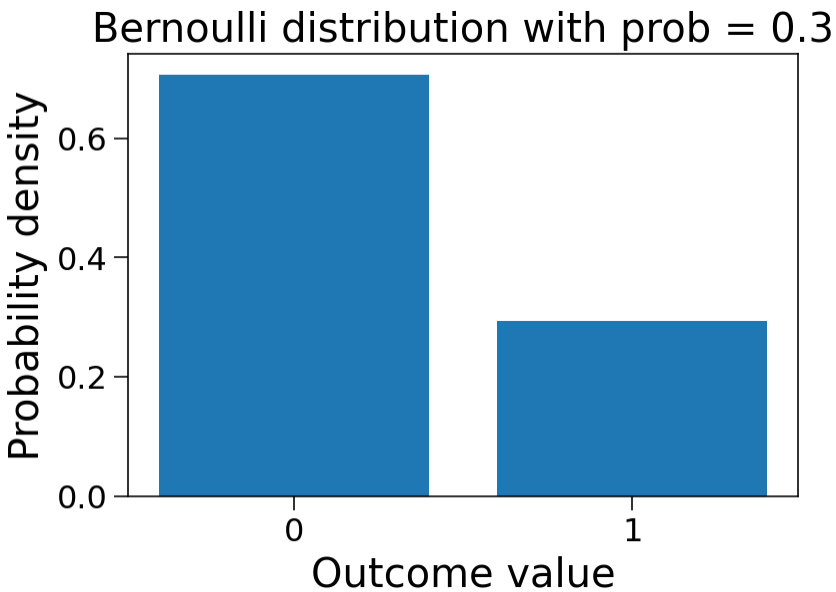
- Sample using `random.expovariate(lambd)`

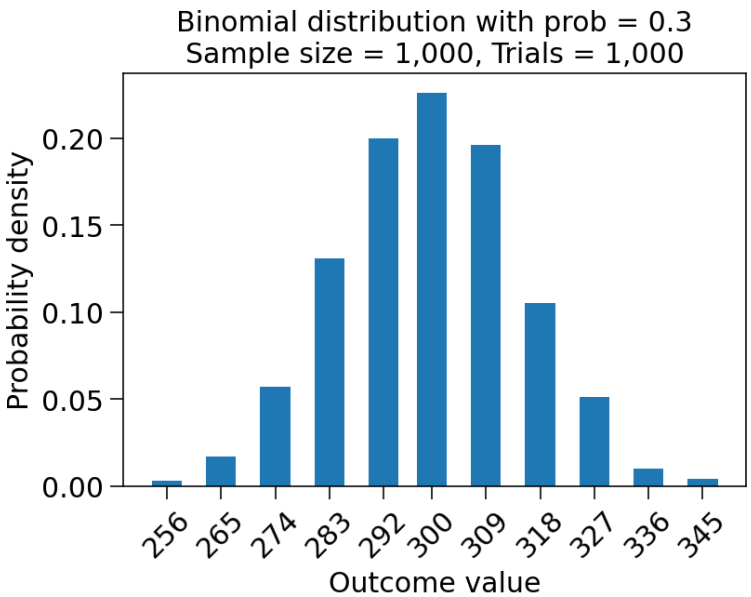
Exponential Distributions: Examples

- Modeling inter-arrival times (time between events), e.g.:
  - Cars entering a highway,
  - Requests for a Web page,
  - Job requests on a server
- Time for a radioactive particle to decay
- Service time of agents in a system (how long a bank teller takes to serve a customer; or waiting times for an airline call center; or waiting time for **help at office hours**)

Bernoulli Distribution

- Describes outcome of a single trial with a 0/1 outcome
- **Binomial distribution** sums multiple trials with binary outcomes





Not an accident that it looks like a normal distribution with mean = 300

Why does this happen?

Galton Board Demo

- Francis Galton invented the “Galton Board” in 1899 to show how a normal distribution is formed by many random events
- As ball hits a post, has equally probability of going left or right; so probability of ending up in any bucket follows binomial distribution
- Let’s try 10 balls
- Let’s try 1,000 balls
- Why normally distributed?

So Far

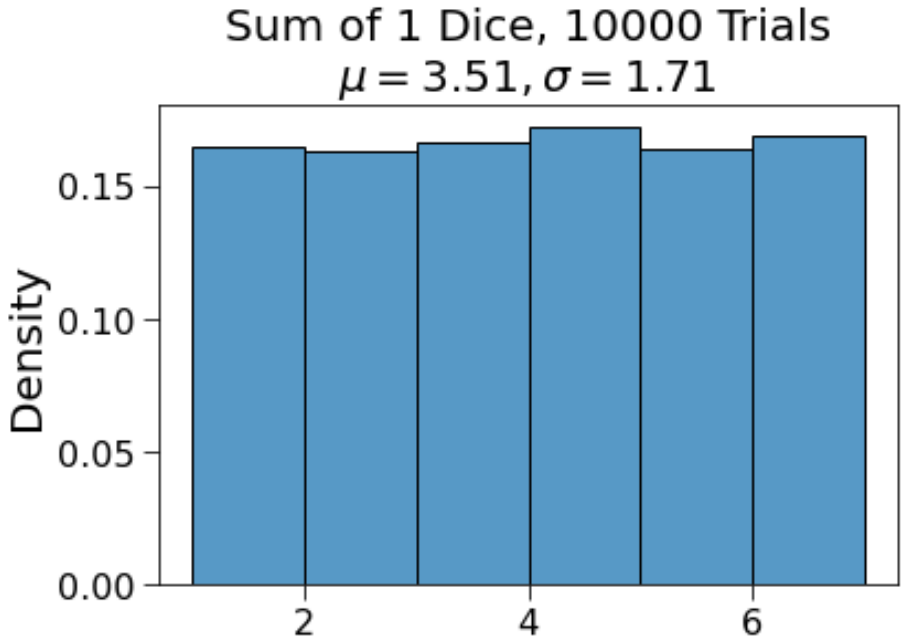
- Showed one way a set of random choices can lead to a predictable distribution
- Now, return to a question we looked at earlier in term: when can we be confident in our estimate of some variable?

Estimating Average Die Roll

- We talked about this earlier, but now we have the tools to analyze it more carefully
- The outcome of a die roll follows a **discrete uniform distribution** on the integers 1 through 6
- Therefore, we know the **mean roll value** (also called the **expected value**) is 3.5
- For the sake of example, let's say we didn't know that, so we estimate it via sampling
- **How many samples** do we need to get a good answer?
- **How confident** can we be in our answer?

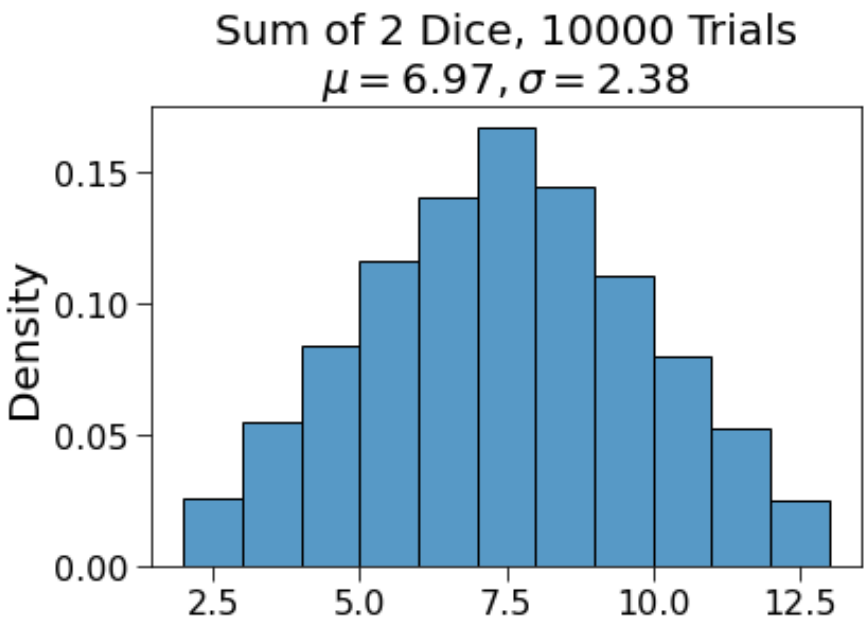
Rolling 1 Die

- Such a sample is drawn directly from a **uniform distribution**, so this is reflected in the empirical distribution of such samples



Rolling 2 Dice and Summing

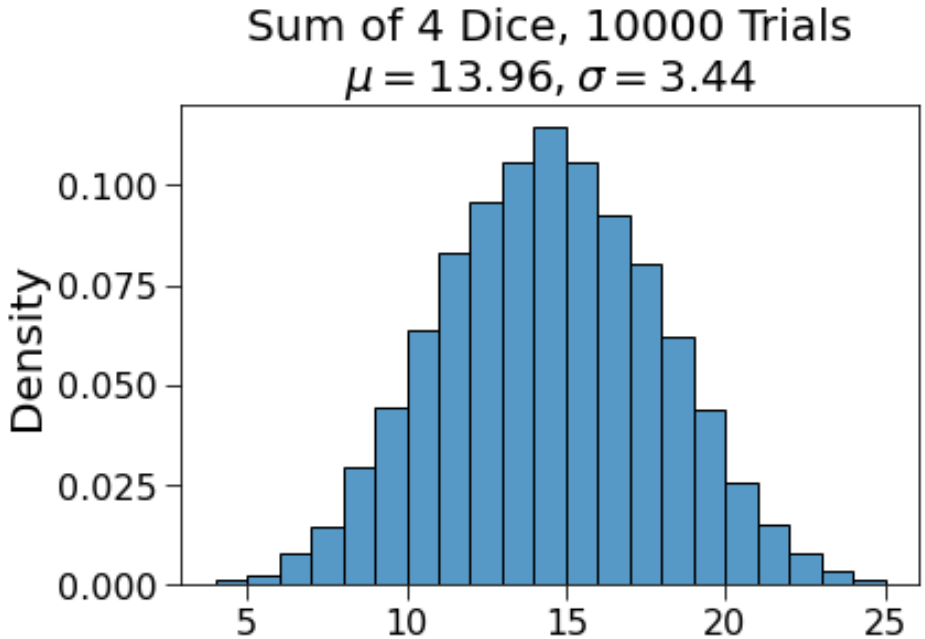
- No longer uniform
- Only one way to form the sums 2 and 12, but six ways to form the sum 7
- In between 2 and 12, the probability linearly increases
- But why did σ go up (from 1.71 to 2.38)?



$CV_1 = 0.49$
$CV_2 = 0.34$

Rolling 4 Dice and Summing

- Peak continues to round out, distribution's edges are starting to flare out
- Distribution looks pretty much like a Gaussian
- While $\boldsymbol{\sigma}$ has been increasing, it doesn't increase as fast as $\boldsymbol{\mu}$, resulting in the distribution becoming narrower



$CV_1 = 0.49$
$CV_2 = 0.34$
$CV_4 = 0.25$

Modeling Dice Rolls with Random Variables

- Each die roll is a uniform random variable

$$D_i \sim \text{Uniform}(1, 6)$$

- The sum of samples $\boldsymbol{D_1}, \ldots, \boldsymbol{D_n}$ is also a random variable

$$S = \sum_{i=1}^{n} D_i$$

- Hence the sample mean is yet another random variable

$$M = \frac{S}{n}$$

- **Claim: $\boldsymbol{S}$, and hence $\boldsymbol{M}$, are normally distributed**
- Which gets us to one of the most important theorems in all of statistics

Central Limit Theorem

Consider a population $\boldsymbol{D}$ with mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma^2}$

1) The **sum** of $\boldsymbol{n}$ independent samples of $\boldsymbol{D}$ tends towards a normal distribution $\boldsymbol{S}$

2) $\boldsymbol{S}$ has mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma^2}$

It's simple, but

IT'S A REALLY BIG DEAL

Central Limit Theorem – Mean version

Consider a population $\boldsymbol{D}$ with mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma^2}$

- The ***mean*** of $\boldsymbol{n}$ independent samples of $\boldsymbol{D}$ tends towards a normal distribution $\boldsymbol{M}$

- $\boldsymbol{M}$ has mean $\boldsymbol{\mu}$ and variance $\frac{\boldsymbol{\sigma^2}}{\boldsymbol{n}}$
  - $\boldsymbol{\sigma_M} = \frac{\boldsymbol{\sigma}}{\sqrt{\boldsymbol{n}}}$

Let's Check It Out

```
print("Empirical values of mean and stdev")
for num_dice in dice:
    dice_means = np.array(roll_dice(num_dice, 6, num_trials)) / num_dice
    mean = np.mean(dice_means)
    stdev = np.std(dice_means)
    print(f'mean of {num_dice:,} rolls, mu={mean:.3f}, sigma={stdev:.3f}')
```

```
print("Theoretical values of mean and stdev, according to CLT")
for num_dice in dice:
    die_mean = sum(range(1,7))/6
    die_variance = (6**2 - 1) / 12 # discrete uniform dist with 6 values
    sum_mean = die_mean * num_dice
    sum_variance = die_variance * num_dice
    mean_mean = sum_mean / num_dice
    mean_variance = sum_variance / num_dice**2
    mean_stdev = mean_variance**0.5
    print(f'mean of {num_dice:,} rolls, mu={mean_mean:.3f}, '
          f'sigma={mean_stdev:.3f}')
```

For a discrete uniform distribution with n possible values, the variance is $(n^2 - 1)/12$

Results

```
Empirical values of mean and stdev, 100 trials
mean of 1 rolls, mu=3.490, sigma=1.628
mean of 5 rolls, mu=3.468, sigma=0.837
mean of 10 rolls, mu=3.503, sigma=0.541
mean of 100 rolls, mu=3.490, sigma=0.162
mean of 1,000,000 rolls, mu=3.500, sigma=0.002

Theoretical values of mean and stdev, according to CLT
mean of 1 rolls, mu=3.500, sigma=1.708
mean of 5 rolls, mu=3.500, sigma=0.764
mean of 10 rolls, mu=3.500, sigma=0.540
mean of 100 rolls, mu=3.500, sigma=0.171
mean of 1,000,000 rolls, mu=3.500, sigma=0.002
```

Recall, $\boldsymbol{M}$ has standard deviation $\boldsymbol{\sigma_M} = \frac{\boldsymbol{\sigma}}{\sqrt{\boldsymbol{n}}}$

As n goes approaches ∞, $\boldsymbol{\sigma_M}$ approaches 0

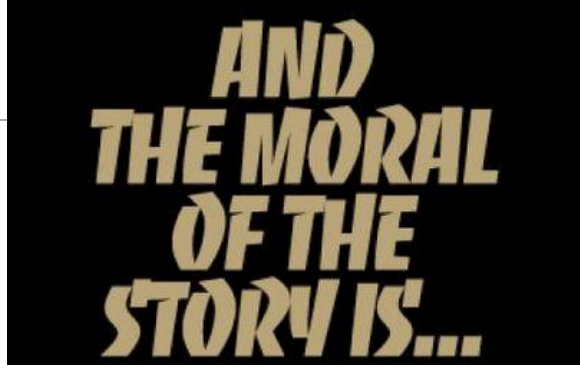
Central Limit Theorem – What Is It Good For?

- The CLT tells us that the distribution, $\boldsymbol{M}$, of the sample means, is (or very well approximates being) normal
- So, we are justified in using the empirical rule to determine a confidence interval based on sampled outcomes $\boldsymbol{m}$ of the random variable $\boldsymbol{M}$

Central Limit Theorem – What Is It Good For?

- Furthermore, the CLT tells us which normal distribution $\boldsymbol{M}$ converges to
  - The mean $\boldsymbol{\mu_M}$ converges to the mean $\boldsymbol{\mu}$ of the population $\boldsymbol{D}$, so a sampled $\boldsymbol{m}$ is an unbiased estimate of $\boldsymbol{\mu}$
  - The standard deviation $\boldsymbol{\sigma_M}$ is known as $\boldsymbol{M}$'s **standard error (or SEM)**
  - It measures how far off a sampled $\boldsymbol{m}$ is expected to be from $\boldsymbol{\mu_M}$
- So, we have all the information needed to calculate a justifiable confidence interval for our estimate of $\boldsymbol{\mu}$ (of $\boldsymbol{D}$)

Summarizing



- It doesn't matter what the shape of the distribution of individual outcomes happens to be
- We can use the Central Limit Theorem to estimate the mean of a population, using a sufficiently large sample
- The Central Limit Theorem also allows us to use the empirical rule when computing confidence intervals associated with an estimated mean

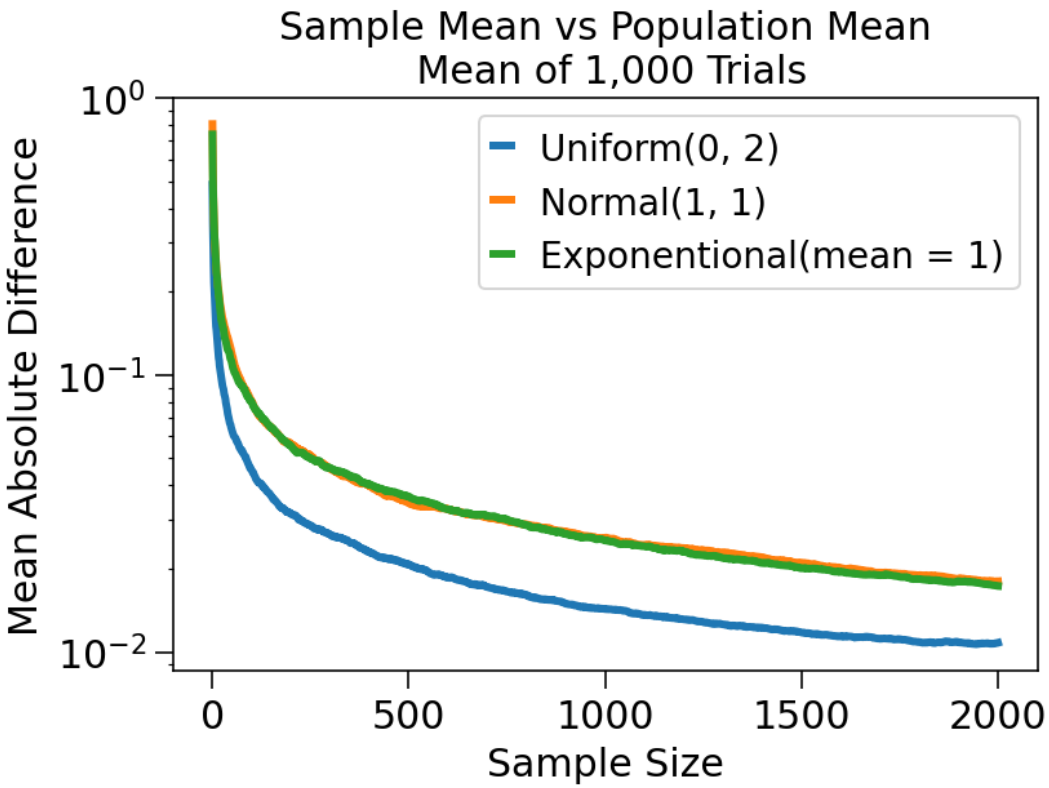
Getting Confidence in Estimate of Mean

- With multiple samples, we can apply the empirical rule
- What if we don’t want to (or can’t) take multiple samples
- How might we approximate this?
- Can we use the standard deviation of one sample

Sample SD vs. Population SD

- Once a sample reaches a “reasonable” size, the sample’s standard deviation is a pretty good approximation to population’s standard deviation
- Does “reasonable” depend on the distribution of the population?
- We can run a computational experiment to find out
  - See posted code

Does Distribution Matter?

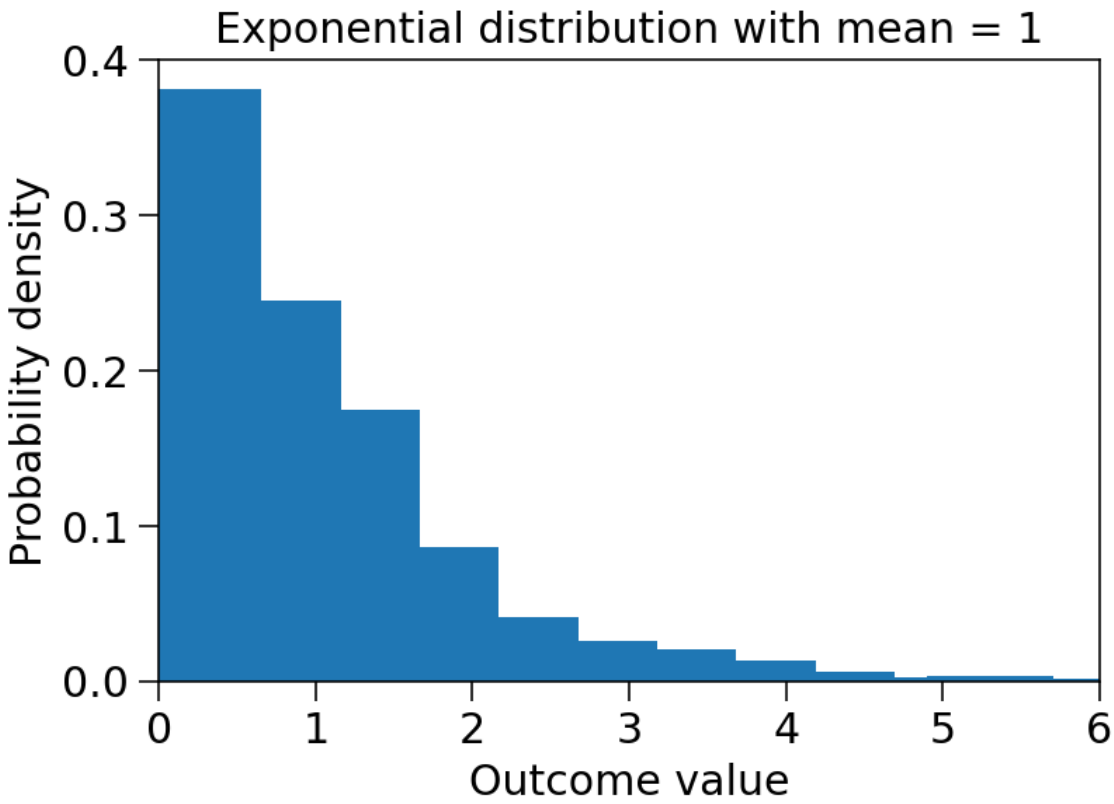


Why does exponential converge slightly faster?

Rate of Convergence Related to Variance in Distribution

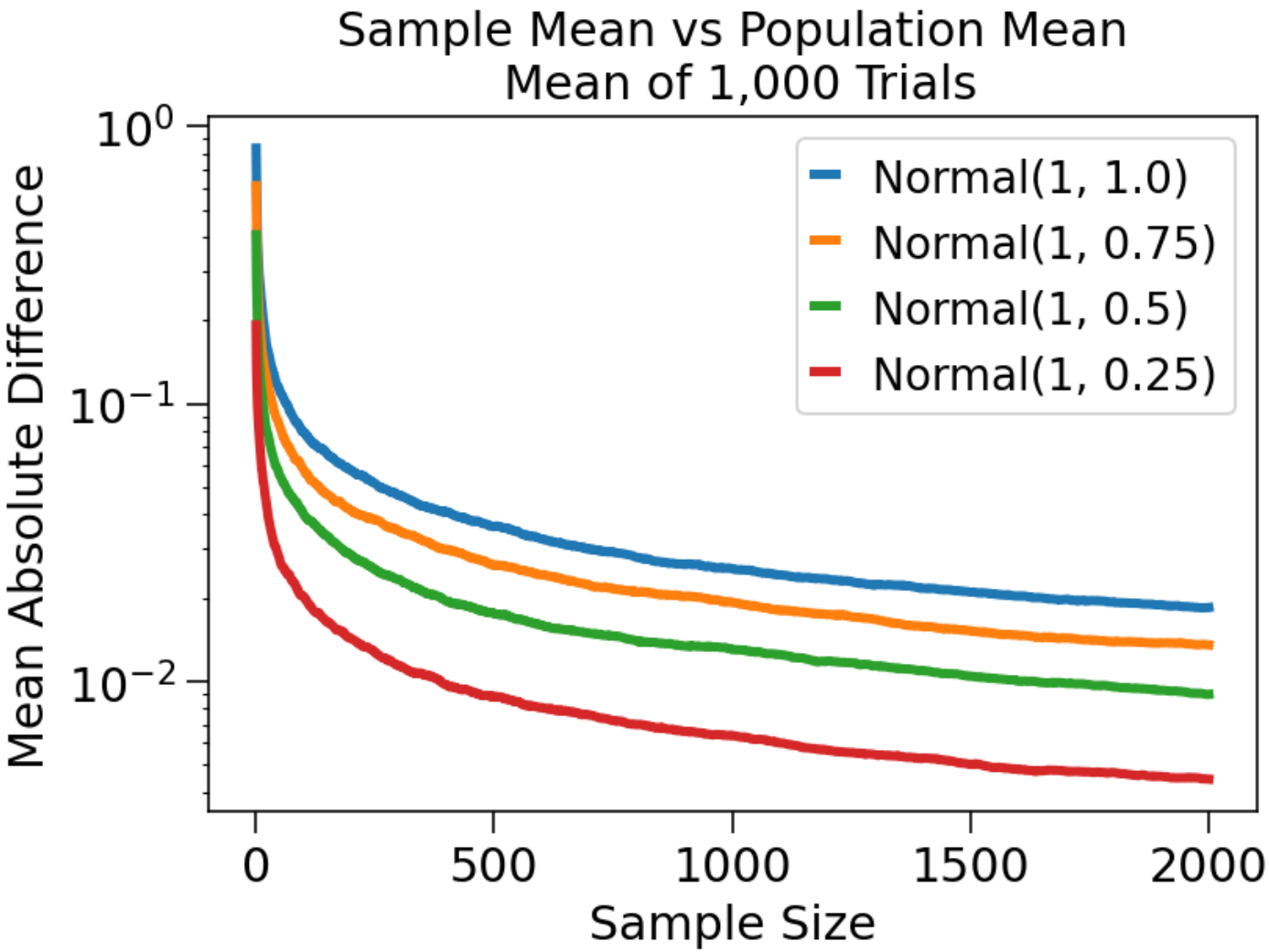
- Variance governed by different parameters for different kinds of distributions
- Uniform: range
- Exponential: lambda
- Normal: sigma

Rate of Convergence Related to Variance in Distribution



Most of the values in the population are close to the mean
So, average of mean of trials likely to be good

The Higher the Variance, the Slower the Convergence



But Suppose We Can Run Only One Trial

1) Choose a sample size
2) Obtain a single random sample of that size from the population
3) Compute the mean and standard deviation of that sample set
4) Calculate the standard error of the sample mean
5) Use the empirical rule to generate a confidence interval around the sample mean for whole population
6) If the interval is too wide, increase the sample size in Step 1

- Doesn't this depend on the population's standard deviation σ?

$$SEM = \frac{\sigma}{\sqrt{n}}$$

One Way to Estimate Mean from a Single Sample

1) Choose a sample size
2) Obtain a single random sample of that size from the population
3) Compute the mean and standard deviation of that sample set
4) Calculate the standard error of the sample mean
   - Use population SD $\boldsymbol{\sigma}$ if available
   - Otherwise use sample set's SD $\boldsymbol{\sigma_{samp}}$
5) Use the Empirical Rule to generate a confidence interval around the sample mean for whole population
6) If the interval is too wide, increase the sample size in Step 1

But wouldn't it be
Much better to start with
a sufficient sample size?

Of course

CLT and Polling

- Assume two candidates, e.g., Jack and Jill
- Randomly poll $\boldsymbol{n}$ people for their opinion (0=Jack, 1=Jill)
  - This is like taking a sample of $\boldsymbol{n}$ coin flips (Bernoulli trials)
  - Average of opinions $\boldsymbol{m} = \boldsymbol{\mu_{samp}}$ is the sample mean
  - Since we are taking a mean of individual Bernoulli outcomes, CLT applies

CLT For Polling

$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$

- Political polls often say things like:
  - "In a poll of 1000 likely voters, Jill is leading Jack by a margin of 54% to 46%, with a margin of error of +/- 3.1% at 95% confidence"
  - CLT is used to derive this bound
- If the poll includes 1000 people and 540 prefer Jill:
  - $\boldsymbol{\mu_{samp}} = 0.54$
  - $\boldsymbol{\sigma^2_{samp}} = \frac{1}{1000}(540 \cdot (1 - 0.54)^2 + 460 \cdot (0 - 0.54)^2)$
  - $\boldsymbol{\sigma_{samp}} = 0.498$
  - SEM = 0.498/1000**0.5 ≈ 0.016 = 1.6%

CLT For Polling

- If $\boldsymbol{\mu_{samp}} = 54\%$, then with **95% confidence**, actual result should be $\mathbf{54\% \pm 3.1\%}$ (3.1% = 1.96*1.6%)
- I.e., **95% of the time the poll is taken**, the computed confidence interval would contain the actual percentage of the population claiming to support Jill

NB: This is an assertion about a poll, not about an election.

How Much Is Enough?

- Can work backwards to determine necessary polling size to achieve certain confidence or accuracy
- E.g., suppose we want to relax our polling accuracy from $\pm\,\mathbf{3.1}\%$ to $\pm\mathbf{5}\%$, while preserving 95% confidence
- **How many people do we need to poll?**

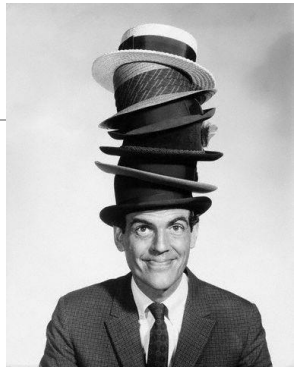
How Much Is Enough?

- For binary polling, the worst case $\boldsymbol{\sigma_{pop}}$ is $\mathbf{0.5}$
  - Half the people vote one way, and half the other
- CLT tells us repeated sample means will be normally distributed with $\boldsymbol{\sigma_M = \sigma_{pop}/\sqrt{n} \leq 0.5/\sqrt{n}}$
- By empirical rule, 95% confidence interval is $\pm\mathbf{1.96}\boldsymbol{\sigma_M}$
- So, if we want a 95% confidence interval to have $< \mathbf{5}\%$ error, we need to solve for $\boldsymbol{n}$ that results in $\mathbf{1.96}\boldsymbol{\sigma_m} < \mathbf{0.05}$

$$\mathbf{1.96} \times \frac{\mathbf{0.5}}{\sqrt{\boldsymbol{n}}} < \mathbf{0.05} \quad \Rightarrow \quad \sqrt{\boldsymbol{n}} > \mathbf{19.6} \quad \Rightarrow \quad \boldsymbol{n} > \mathbf{384}$$

NB: the size of the population doesn't matter!

Recapping



- Understanding populations
  - Cannot examine all members
  - Rely on sampling
  - Estimating validity of conclusions based on samples
  - Central Limit Theorem lets us use a single sample, and still assert inferences with specific confidence levels
- Some math, but goal was to use computation to help develop intuition
- Now for something completely different

Pandas

Python Data Science Stack

Goal of this Part of Lecture

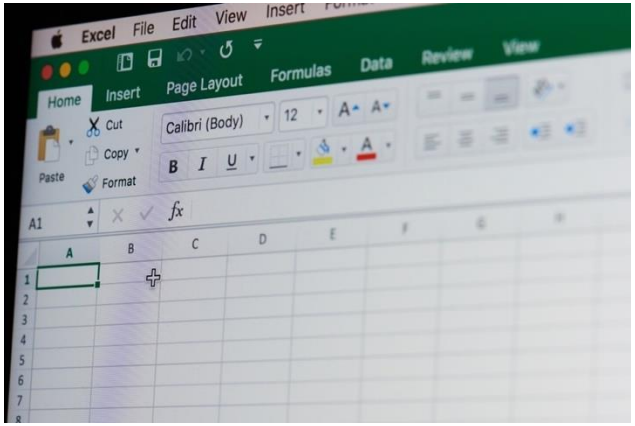
- Generally, try to teach principles, not tools
- But some tools are widely used and illustrative of larger ideas
- Expectations
  - Don't expect full ability to use it based on lecture
  - ***Don't sweat the details, do sweat the concepts***

- You will want to use Pandas on next problem set
  - Make good use of PyTutor

Break

Pandas

- Provides mechanisms for
  - Organizing data
  - Calculating simple statistics about data
  - Storing data in formats that facilitate future analysis
- Far from elegant, but handy and widely used
- Key structure is the **DataFrame**
  - Mutable two-dimensional tabular data structure



Building DataFrames

- Construct from scratch using Python
  - Don’t expect to get to it today; there are some slides on it
- Read from a CSV (comma separated values) file
  - Been hand coding functions to do this all term
  - Pandas makes it easy

```
Round,Winner,W Goals,Loser,L Goals
Quarters,Spain,2,Netherlands,1
Quarters,Sweden,2,Japan,1
Quarters,England,2,Columbia,1
Quarters,Australia,0,France,0
Semis,Spain,2,Sweden,1
Semis,England,3,Australia,1
3rd Place,Sweden,2,Australia,0
Championship,Spain,1,England,0
```

Getting a DataFrame from a CSV

```
import pandas as pd
```

```
wwc = pd.read_csv('wwc2022_qf.csv')
print(wwc)
```

Index (row) label

Column label

Columns axis = 1

Rows axis = 0

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

Accessing Contents

Index (row) label

Column label

Columns axis = 1

Rows axis = 0

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

- Access contents using attributes `index`, `columns`, `values`

Accessing Contents: Getting Column Labels

Index (row) label

Column label

Columns axis = 1

Rows axis = 0

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

```
print(wwc.columns)
```

Index(['Round', 'Winner', 'W Goals', 'Loser', 'L Goals'], dtype='object')

Type pandas.Index

- **Immutable sequence** of labels
- Optimized for **fast lookup**

Accessing Contents: Index

Index (row) label

Column label

Columns axis = 1

Rows axis = 0

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

```
print(wwc.columns)
print(wwc.index)
```

Index(['Round', 'Winner', 'W Goals', 'Loser', 'L Goals'], dtype='object')
RangeIndex(start=0, stop=8, step=1)

Accessing Contents: Values of Cells

Index (row) label

Column label

Columns axis = 1

Rows axis = 0

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

```
print(wwc.values)
print(type(wwc.values))
```

```
[['Quarters' 'Spain' 2 'Netherlands' 1]
 ['Quarters' 'Sweden' 2 'Japan' 1]
 ['Quarters' 'England' 2 'Columbia' 1]
 ['Quarters' 'Australia' 0 'France' 0]
 ['Semis' 'Spain' 2 'Sweden' 1]
 ['Semis' 'England' 3 'Australia' 1]
 ['3rd Place' 'Sweden' 2 'Australia' 0]
 ['Championship' 'Spain' 1 'England' 0]]
<class 'numpy.ndarray'>
```

Type is numpy.ndarray
an array of arrays with
shape = (8, 5)
8 rows, 5 columns

Selecting Columns

- Square brackets primary mechanism
- Selection returns a series, not a DataFrame
  - Can treat as an iterable

```
winners = ''
for w in wwc['Winner']:
    winners += w + ', '
print(winners[:-2])
```

Spain, Sweden, England, Australia, Spain, England, Sweden, Spain

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

Selecting Rows

```
row_2 = ''
for r in (wwc.iloc[2]):
    row_2 += str(r) + ', '
print(row_2[:-2])
```

Quarters, England, 2, Columbia, 1

Why not `wwc[2]`?

Indexing Pandas defaults to label based, not position based

`iloc` tells it to use position

Many, many other selection mechanisms

	Round	Winner	W Goals	Loser	L Goals
0	Quarters	Spain	2	Netherlands	1
1	Quarters	Sweden	2	Japan	1
2	Quarters	England	2	Columbia	1
3	Quarters	Australia	0	France	0
4	Semis	Spain	2	Sweden	1
5	Semis	England	3	Australia	1
6	3rd Place	Sweden	2	Australia	0
7	Championship	Spain	1	England	0

A More Interesting Example

```
temps = pd.read_csv('US_temperatures.csv')
print(temps)
```

```
           Date  Albuquerque  ...  St Louis  Tampa
0      19610101        -0.55  ...     -0.55  15.00
1      19610102        -2.50  ...     -0.55  13.60
2      19610103        -2.50  ...      0.30  11.95
...         ...          ...  ...       ...    ...
20085  20151229        -2.15  ...      1.40  26.10
20086  20151230        -2.75  ...      0.60  25.55
20087  20151231        -0.75  ...     -0.25  25.55

[20088 rows x 22 columns]
```

Making It Easier for Me to Read

```
# Make date the index, reformat dates, change temp to F
temps['Date'] = pd.to_datetime(temps['Date'], format='%Y%m%d')
temps.set_index('Date', inplace = True)
temps[temps.columns] = temps[temps.columns]*9/5 + 32
print(temps)
```

	Albuquerque	Baltimore	...	Min T	Mean T
Date			...		
1961-01-01	31.01	38.03	...	23.45	44.35
1961-01-02	27.50	32.00	...	23.00	41.52
1961-01-03	27.50	33.53	...	12.56	40.10
...	...	...	...	...	...
2015-12-29	28.13	49.01	...	28.13	48.68
2015-12-30	27.05	49.46	...	27.05	48.81
2015-12-31	30.65	49.55	...	25.16	48.72

Making It Easier for Me to Read

```
# Make date the index, reformat dates, change temp to F
temps['Date'] = pd.to_datetime(temps['Date'], format='%Y%m%d')
temps.set_index('Date', inplace = True)
temps[temps.columns] = temps[temps.columns]*9/5 + 32
print(temps)
```

Making It Easier for Me to Read

```
# Make date the index, reformat dates, change temp to F
temps['Date'] = pd.to_datetime(temps['Date'], format='%Y%m%d')
temps.set_index('Date', inplace = True)
temps[temps.columns] = temps[temps.columns]*9/5 + 32
print(temps)
```

	Albuquerque	Baltimore	Boston	...	Seattle	St Louis	Tampa
Date			...				
1961-01-01	31.01	38.03	35.51	...	37.58	31.01	59.00
1961-01-02	27.50	32.00	30.56	...	32.99	31.01	56.48
1961-01-03	27.50	33.53	26.51	...	32.00	32.54	53.51
1961-01-04	29.57	27.50	25.43	...	40.01	40.46	48.02
1961-01-05	31.55	29.48	23.99	...	47.03	44.96	52.97
...	...	...	...	...	...	...	
2015-12-27	25.61	59.00	43.97	...	37.49	42.44	77.99
2015-12-28	23.72	46.49	33.53	...	38.03	42.53	77.99
2015-12-29	28.13	49.01	32.09	...	39.02	34.52	78.98
2015-12-30	27.05	49.46	31.10	...	36.14	33.08	77.99
2015-12-31	30.65	49.55	40.01	...	35.15	31.55	77.99

Easy to Look at Specific Dates/Cities

```
print(temps.loc['2015-12-31'].to_string())
```

```
Albuquerque      30.65
Baltimore        49.55
Boston           40.01
Charlotte        59.54
Chicago          25.16
Dallas           43.07
Detroit          33.53
Las Vegas        40.01
Los Angeles      55.49
Miami            79.52
New Orleans      57.56
New York         47.48
Philadelphia     47.03
Phoenix          48.56
Portland         36.05
San Diego        54.95
San Francisco    45.05
San Juan         77.99
Seattle          35.15
St Louis         31.55
Tampa            77.99
```

Look at Subset of Cities on a Date

```
print(temps.loc['2015-12-31', ['Boston', 'Tampa']])
```

```
Boston    40.01
Tampa     77.99
```

Some Statistics for Each Date

```
temps['Max T'] = temps.max(axis = 'columns')
temps['Min T'] = temps.min(axis = 'columns')
temps['Mean T'] = temps.mean(axis = 'columns').round(2)
print(temps)
```

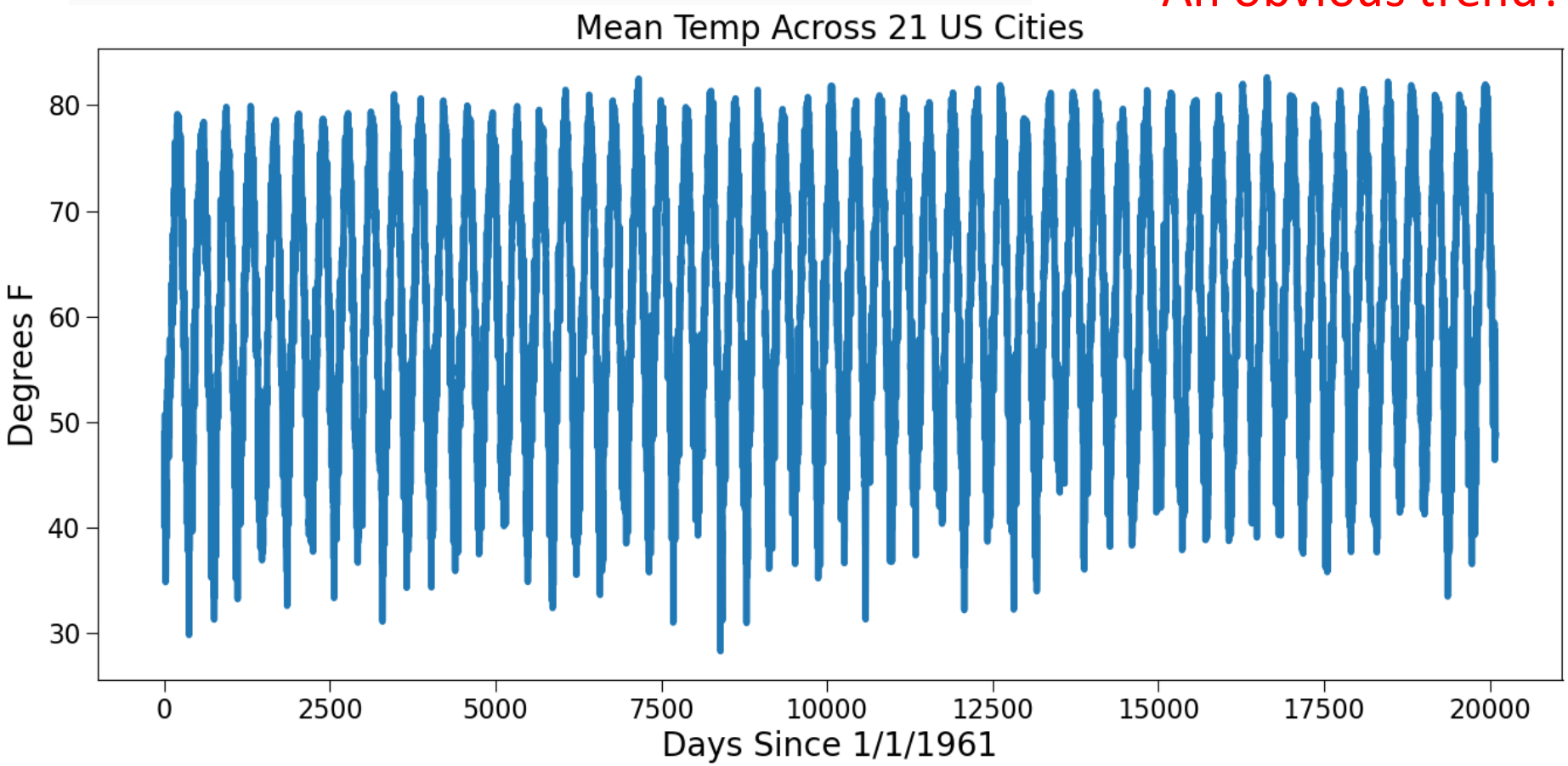
```
            Albuquerque  Baltimore  ...  Min T  Mean T
Date                                ...
1961-01-01        31.01      38.03  ...  23.45   44.35
1961-01-02        27.50      32.00  ...  23.00   41.52
1961-01-03        27.50      33.53  ...  12.56   40.10
...                 ...        ...  ...    ...     ...
2015-12-29        28.13      49.01  ...  28.13   48.68
2015-12-30        27.05      49.46  ...  27.05   48.81
2015-12-31        30.65      49.55  ...  25.16   48.72

[20088 rows x 24 columns]
```

Summarize country for each date

Let's Plot the Mean

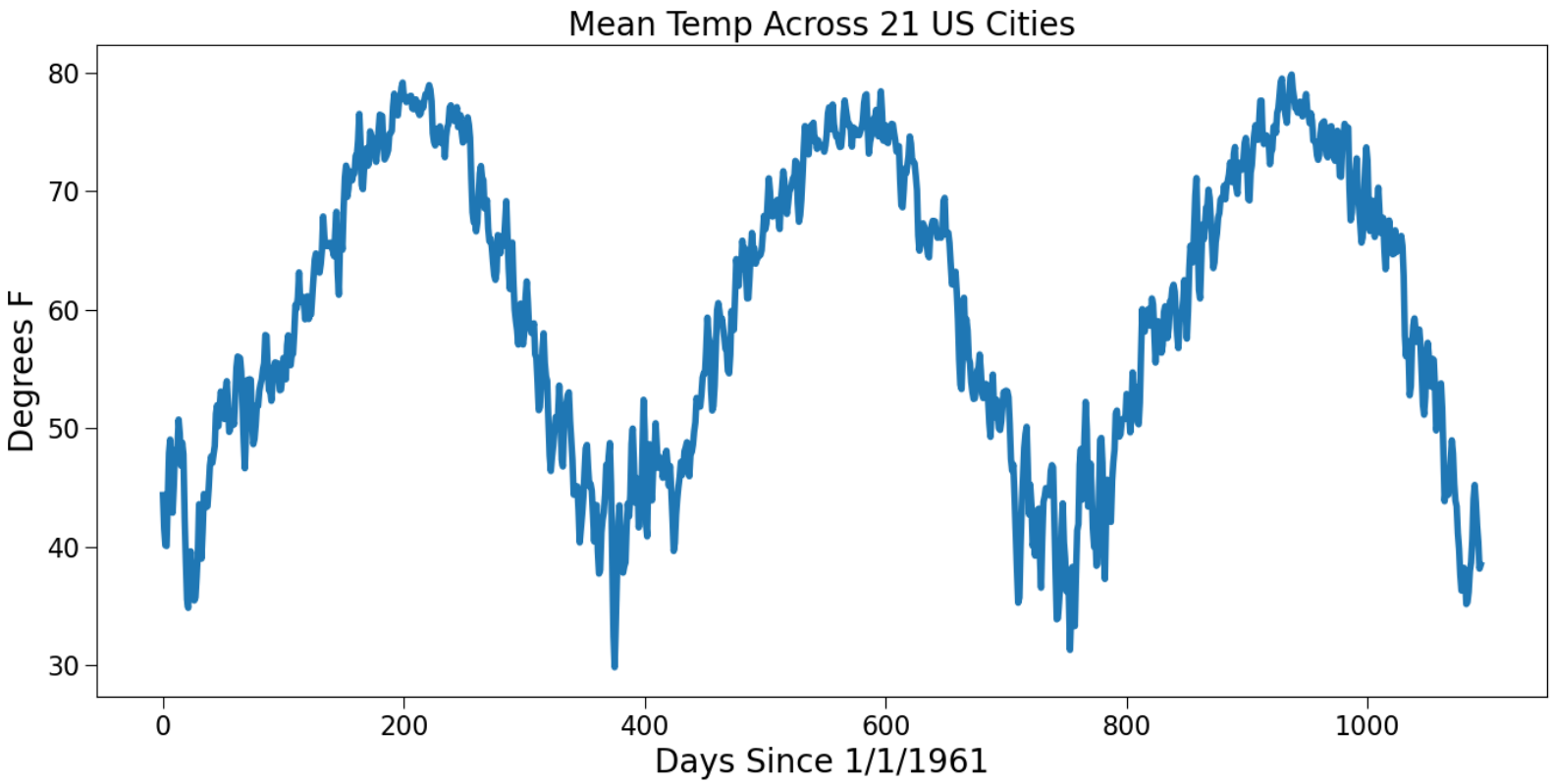
```
plt.figure(figsize = (14, 5))
plt.plot(list(temps['Mean T']))
plt.title('Mean Temp Across 21 US Cities')
plt.xlabel('Days Since 1/1/1961')
plt.ylabel('Degrees F')
```

An obvious trend?

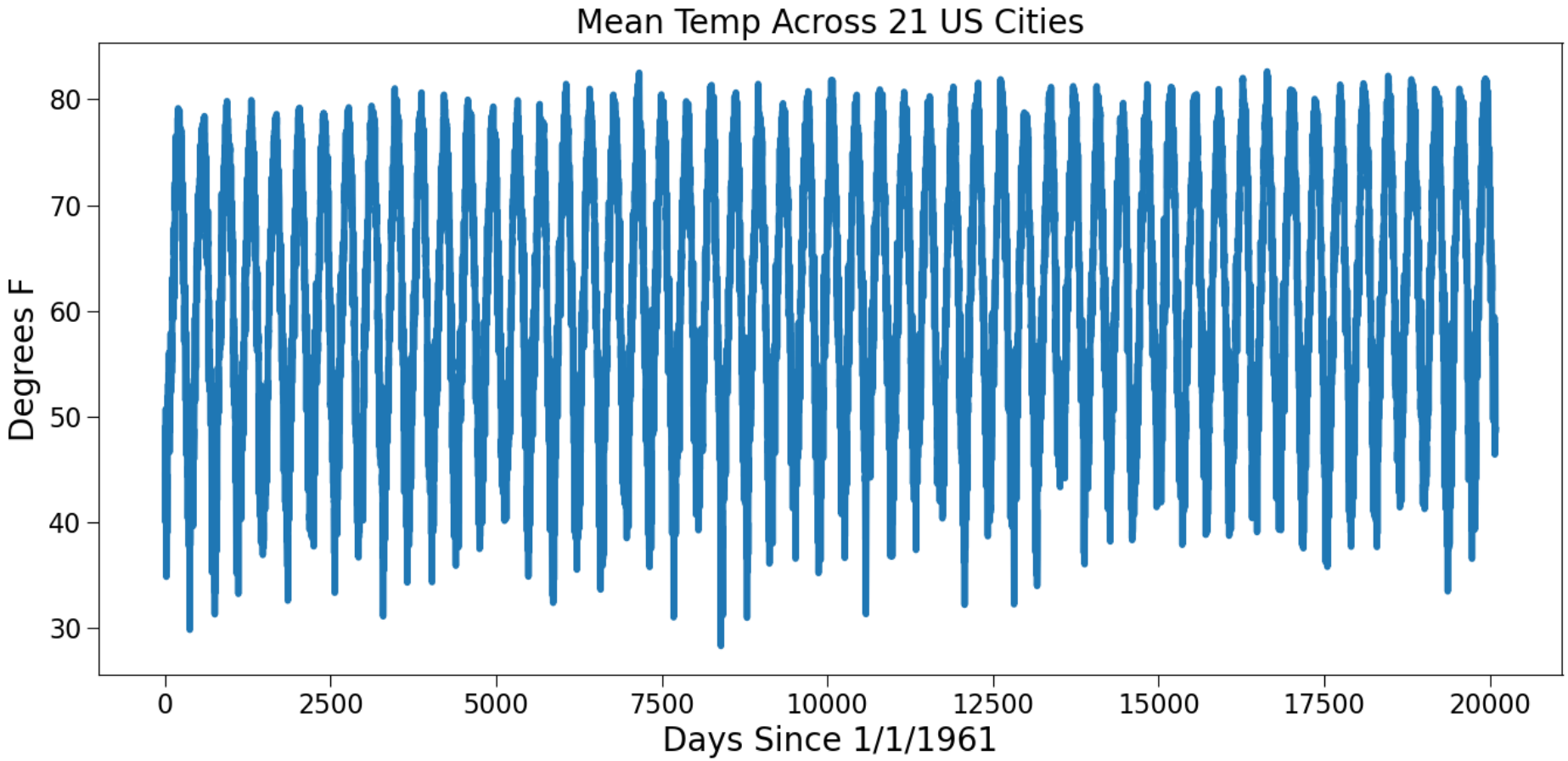


Obvious Short-term Trend If We Zoom In

```
plt.plot(list(temps['Mean T'])[0:3*365])
```



But Long-term Trends Not So Obvious



Let's look at annual rather than daily data

Getting Daily Averages

```
temps.reset_index(drop = False, inplace = True)
temps['Year'] = temps['Date'].apply(lambda d: str(d)[0:4])
temps = temps.loc[:, ['Year', 'Min T', 'Max T', 'Mean T']]
print(temps)
```

Moves the current index into a regular column.

Changes the value in the new column to include just year

By applying a function to each cell

Creates a DataFrame will all rows (:) and selected columns

```
       Year  Min T  Max T  Mean T
0      1961  23.45  76.01   44.35
1      1961  23.00  77.00   41.52
2      1961  12.56  76.55   40.10
...     ...    ...    ...     ...
20085  2015  28.13  79.97   48.68
20086  2015  27.05  80.51   48.81
20087  2015  25.16  79.52   48.72

[20088 rows x 4 columns]
```

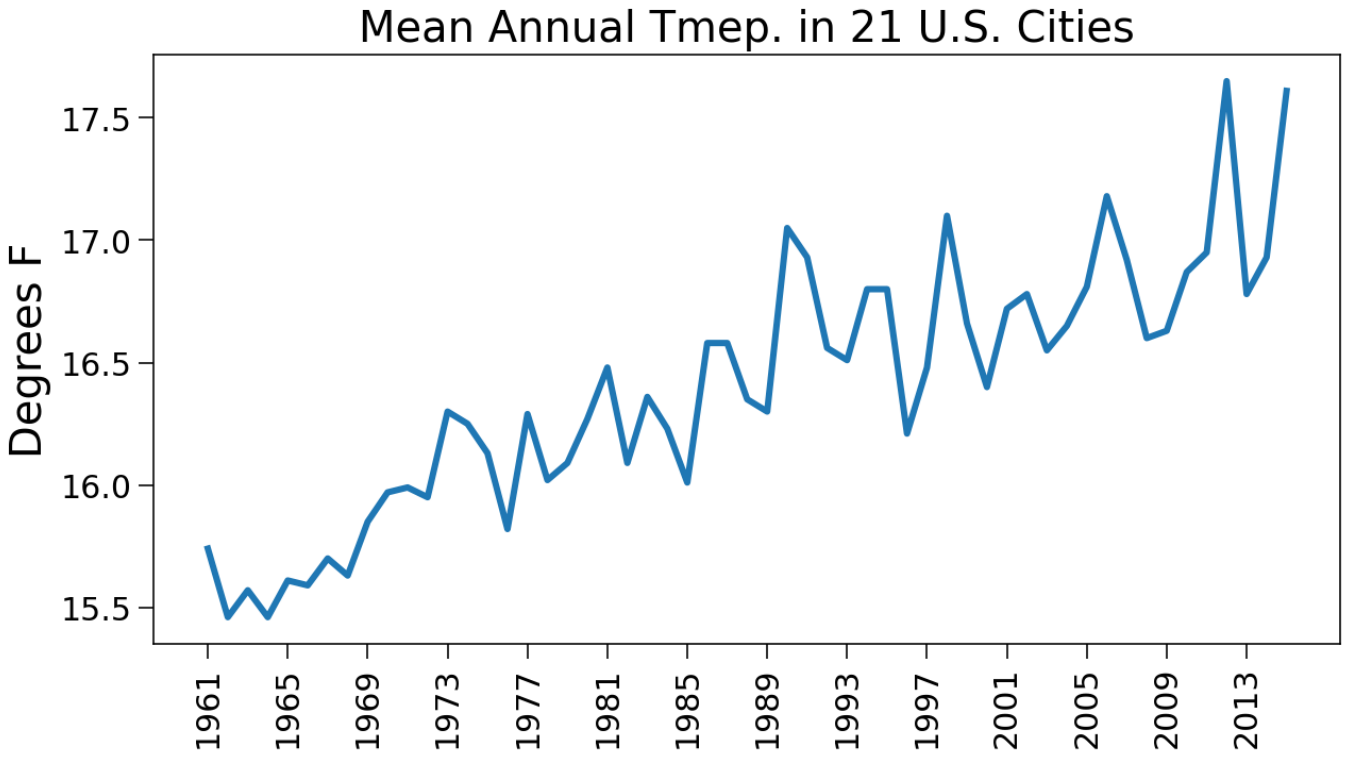
Getting Means Across Years

```
grouped_temps = temps.groupby('Year').mean()
print(grouped_temps)
```

```
           Min T      Max T     Mean T
Year
1961  42.502877  81.704658  60.326767
1962  40.858219  81.519233  59.826438
1963  40.669836  81.953699  60.026411
...         ...        ...        ...
2013  43.415205  85.488603  62.197370
2014  42.695205  85.546301  62.477425
2015  44.174658  85.710521  63.698575

[55 rows x 3 columns]
```

Producing an Informative Plot

```
years = [str(yr) for yr in range(1961, 2016)]
yearly_means = [round(grouped_temps.loc[yr]['Mean T'].mean(), 2)
                for yr in years]
plt.figure(figsize = (10, 5))
plt.plot(years, yearly_means)
plt.title('Mean Annual Tmep. in 21 U.S. Cities')
plt.ylabel('Degrees F')
plt.xticks(range(0, len(years), 4), rotation = 'vertical')
```



Looking at Min and Max Temps By City

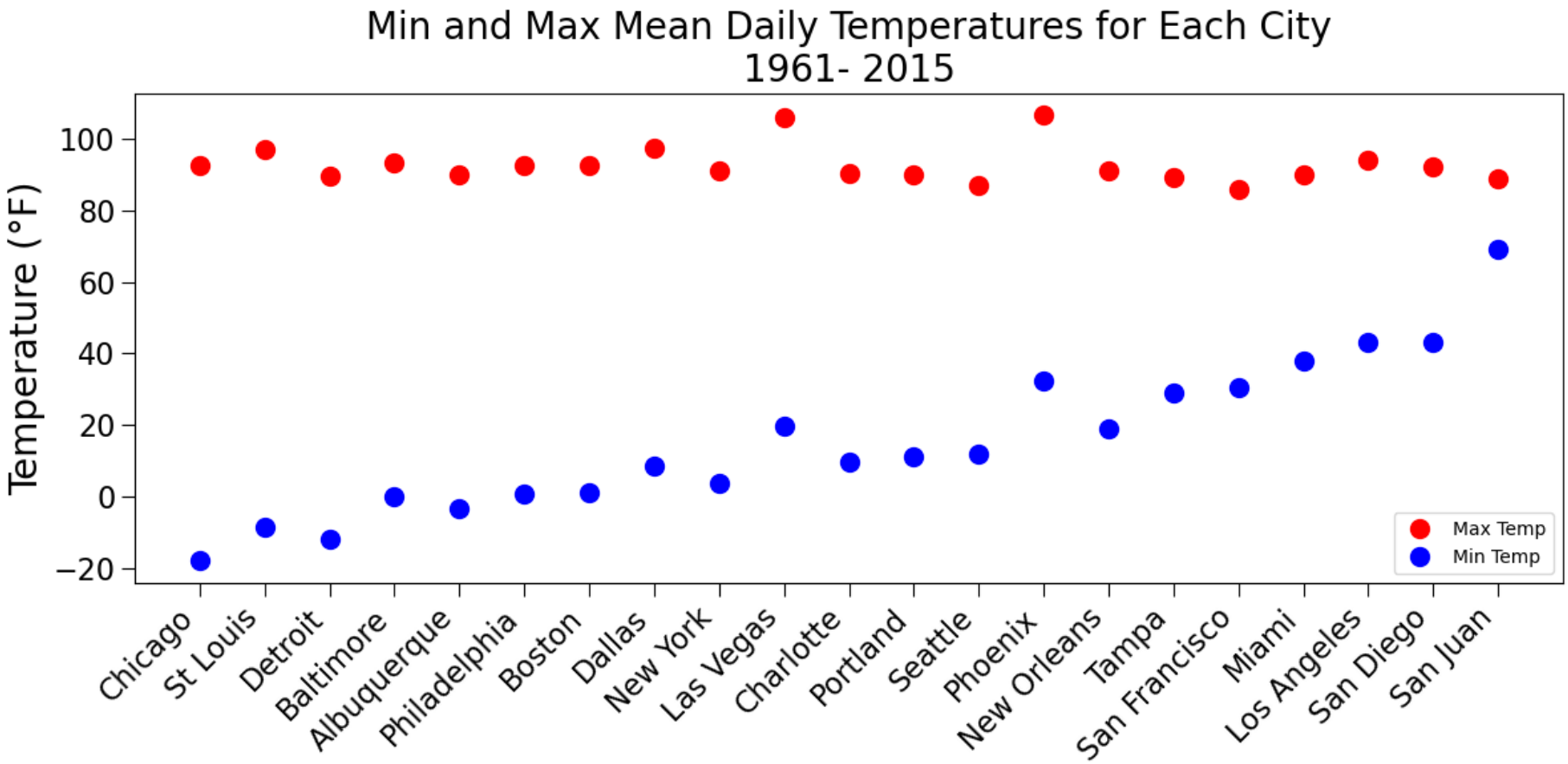
```
temps = pd.read_csv('US_temperatures.csv')
temps.drop('Date', axis = 'columns', inplace = True)
temps[temps.columns] = temps[temps.columns]*9/5 + 32
min_max = pd.DataFrame({'Min': temps.min(), 'Max': temps.max()})
print(min_max)
```

```
                Min    Max
Albuquerque   -3.46  89.96
Baltimore     -0.04  93.47
Boston         1.04  92.48
...             ...    ...
Seattle       12.02  86.99
St Louis      -8.50  96.98
Tampa         28.94  89.06

[21 rows x 2 columns]
```

Let's Plot the Data

```
min_max['Range'] = min_max['Max'] - min_max['Min']
min_max_sorted = min_max.sort_values('Range', ascending=False)
plt.figure(figsize=(12,6))
plt.plot(min_max_sorted.index, min_max_sorted['Max'], 'ro',
         label='Max Temp')
plt.plot(min_max_sorted.index, min_max_sorted['Min'], 'bo',
         label='Min Temp')
plt.xticks(rotation=45, ha='right')
plt.ylabel('Temperature (°F)')
plt.title('Min and Max Mean Daily Temperatures for Each City' +
          '\n1961- 2015')
plt.legend(loc = 'lower right')
```

Where Would You Choose to Live?



What is the relationship between the min and max temperatures?

What is the correlation?

Correlation

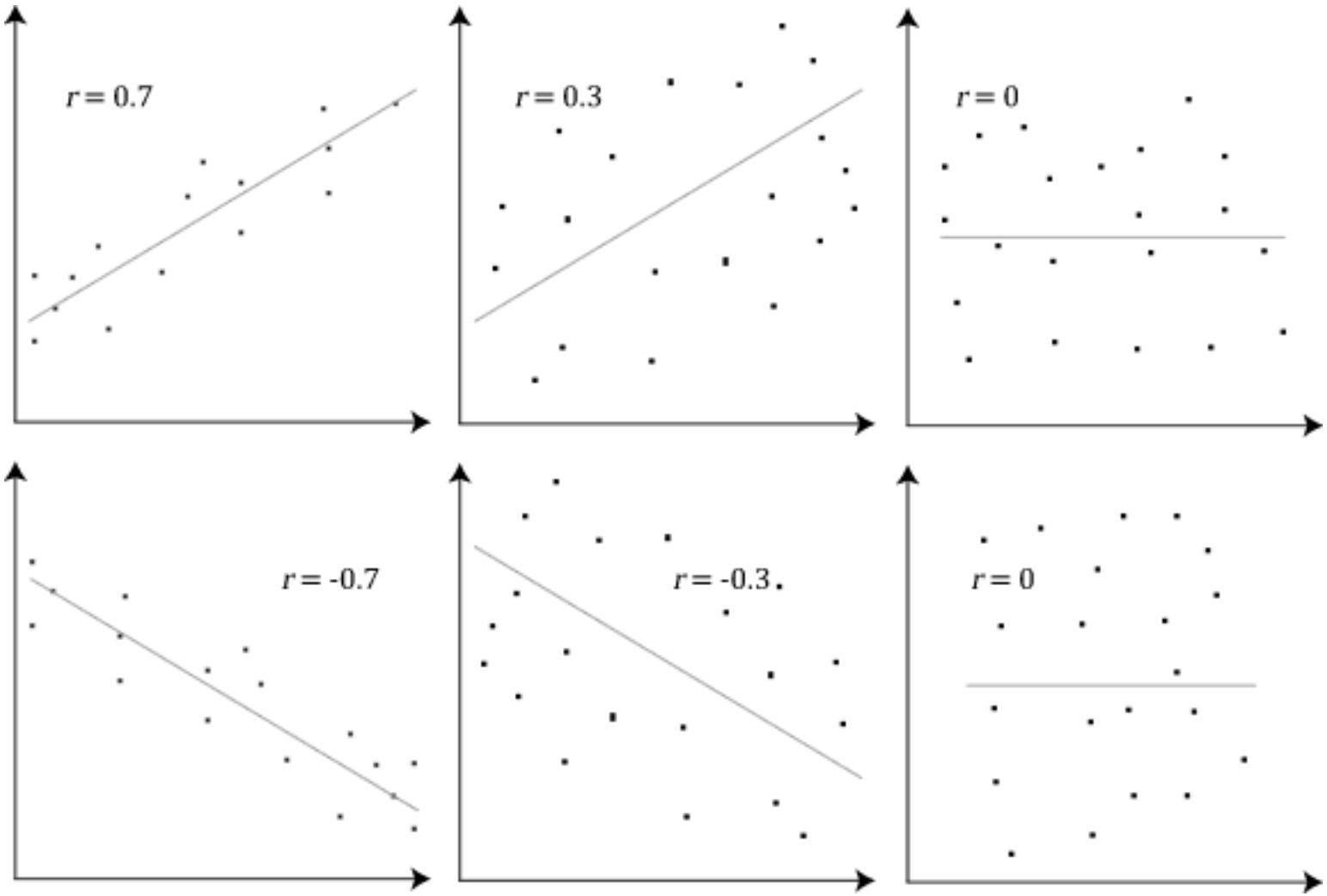
- Do the points in two series, X (mins) and Y (maxes), move in the same direction?
- +1: Perfect positive correlation, whenever X increases, Y increases
- 0: No linear relationship between X and Y
- -1: Perfect negative correlation, whenever X increases, Y decreases

$$\mathrm{cor}(X, Y) = \frac{\mathrm{cov}(X, Y)}{\sigma X * \sigma Y}$$

$$\mathrm{cov}(X, Y) = \frac{\sum_{i=1}^{n}(X_i - \mu X)(Y_i - \mu Y)}{n - 1}$$

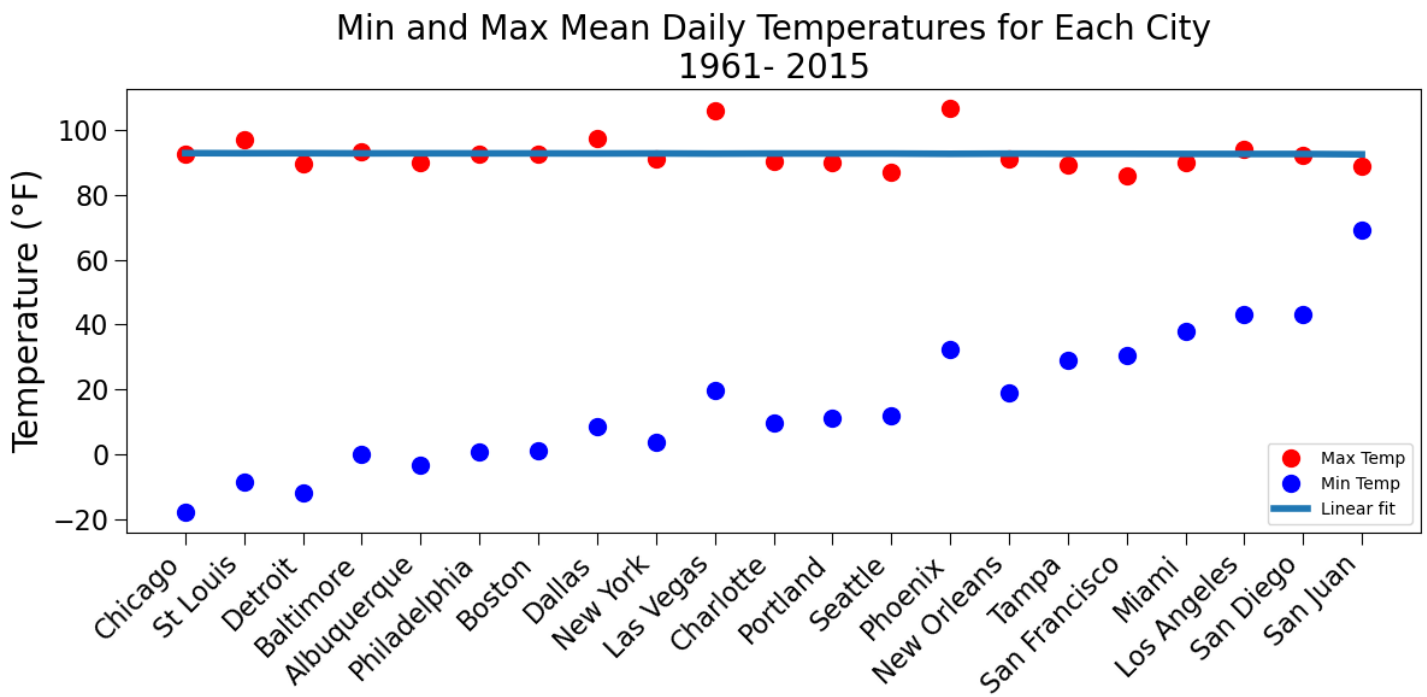
Covariance

Correlation Between X and Y Values



Correlation Is Nearly Zero

```
model_1 = np.polyfit(min_max_sorted['Min'], min_max_sorted['Max'], 1)
print(f'model = {round(model_1[0], 3)}*x + {round(model_1[1], 3)}')
preds = np.polyval(model_1, min_max_sorted['Min'])
plt.plot(preds, label = 'Linear fit')
plt.legend()
r2 = round(r_squared(min_max_sorted['Max'], preds), 6)
print(f'r-squared = {r2}')
cor = round(min_max_sorted['Min'].corr(min_max_sorted['Max']),6)
print(f'correlation = {cor}')
```



correlation = -0.017389 model = -0.004*x + 92.778 r-squared = 0.000302

Take-home Message

- Pandas allows us to interact with data as a "programmatic spreadshseet"
  - Operates on DataFrames and series
- It a powerful library
  - Barely scratched the surface today
  - Never necessary, but can simplify lots of tasks
- List of available ways to learn more
  - Textbook
  - https://www.dataschool.io/best-python-pandas-resources/
- You will want to use it on next problem set
- You will not be asked to write Pandas code on quiz

Extra Slides

Creating a DataFrame Using Code

```
print(pd.DataFrame())
rounds = ['Semis', 'Semis', '3rd Place', 'Championship']
teams = ['Spain', 'England', 'Sweden', 'Spain']
df = pd.DataFrame({'Round': rounds, 'Winner': teams})
print(df)
```

Empty DataFrame

A dictionary

```
Empty DataFrame
Columns: []
Index: []
```

```
           Round   Winner
0          Semis    Spain
1          Semis  England
2      3rd Place   Sweden
3  Championship    Spain
```

Adding a Column

```
df['W Goals'] = [2, 3, 2, 1]
print(df)
```

```
           Round   Winner  W Goals
0          Semis    Spain        2
1          Semis  England        3
2      3rd Place   Sweden        2
3  Championship    Spain        1
```

Mutates df

Deleting a Column

```
df_new = df.drop('Winner', axis = 'columns', inplace = False)
print(df_new, '\n')
print(df, '\n')
```

No mutation

```
           Round  W Goals
0          Semis        2
1          Semis        3
2      3rd Place        2
3  Championship         1
```

```
           Round   Winner  W Goals
0          Semis    Spain        2
1          Semis  England        3
2      3rd Place   Sweden        2
3  Championship    Spain        1
```

```
df.drop('Winner', axis = 'columns', inplace = True)
print(df)
```

Mutation

```
           Round  W Goals
0          Semis        2
1          Semis        3
2      3rd Place        2
3  Championship         1
```

Add Winner Column Back

```
df['Winner'] = teams
print(df)
```

```
          Round  W Goals   Winner
0         Semis        2    Spain
1         Semis        3  England
2     3rd Place        2   Sweden
3  Championship        1    Spain
```

Adding Rows

Create a dict where each entry corresponds to a column

- Key is name of column
- Value a list of the value of that column for each row

```
quarters_dict = {'Round': ['Quarters'] * 4,
                 'Winner': ['Spain', 'Sweden', 'England', 'Australia'],
                 'W Goals': [2, 2, 2, 0]}
```

Convert that dict into a DataFrame and then concatenate it with existing DataFrame

```
df = pd.concat([pd.DataFrame(quarters_dict), df], sort = False)
```

Sort tells it whether to sort the columns lexicographically

Adding the Quarter Finals Winners

```
quarters_dict = {'Round': ['Quarters'] * 4,
                 'Winner': ['Spain', 'Sweden', 'England', 'Australia'],
                 'W Goals': [2, 2, 2, 0]}
df = pd.concat([pd.DataFrame(quarters_dict), df], sort = False)
print(df.to_string())
```

```
           Round     Winner  W Goals
0       Quarters      Spain        2
1       Quarters     Sweden        2
2       Quarters    England        2
3       Quarters  Australia        0
0          Semis      Spain        2
1          Semis    England        3
2      3rd Place     Sweden        2
3   Championship      Spain        1
```

Anything you don't like about this?

Resetting the Indices

```
df = pd.concat([pd.DataFrame(quarters_dict), df], sort = False,
               ignore_index = True)
```

	Round	Winner	W Goals
0	Quarters	Spain	2
1	Quarters	Sweden	2
2	Quarters	England	2
3	Quarters	Australia	0
4	Semis	Spain	2
5	Semis	England	3
6	3rd Place	Sweden	2
7	Championship	Spain	1